01 先认识：你的算法为什么需要四个“调节旋钮”？

阅读目标：看懂当前实现，不要求你先学会优化理论。本文不修改论文或代码；所有手算数字都明确是教学例子，不是实验结果。核对日期：2026 年 10 月 31 日。

你希望调度算法**尽量省计算能耗**，但不能为了省电，把工作流拖得太晚，或让它的模型可靠性低于要求。因此，“省电”和“守约束”要一起考虑。

把乘子理解为“违反某类要求时，要扣多少分”的系数。**代价**表示这件事有多糟；**乘子**决定这类糟糕事情有多值得重视。两者相乘才是惩罚。

你的两层策略做不同的事，所以有四个独立的系数：

乘子	对应的问题	作用位置
μ_D^H	上层选服务器，要多重视整体 Deadline 代价？	上层动作评分
μ_R^H	上层选服务器，要多重视可靠性代价？	上层动作评分
μ_D^L	下层调频，要多重视降频增加的迟到？	下层训练奖励
μ_R^L	下层调频，要多重视可靠性代价？	下层训练奖励

符号不要混：

- 上标 H 表示上层，L 表示下层；下标 D 表示 Deadline，R 表示可靠性。它们是标签，不是乘方。
- w：一个工作流；i：一个任务；e：一整批工作流形成的训练轨迹。
- t：批内的一次任务调度决策；k：控制器的一次有效更新。k 不是任务编号。
- J：控制器最近观测到的统计量；b：允许的预算；$J-b$：超预算多少。
- 本文用 U 表示“记住过去误差的累积量”。U 与真正使用的乘子 μ 不是同一个变量。

学习顺序：先读第 2–3 页理解“为什么扣分”，再读第 4–8 页手算 Deadline 控制器，最后读可靠性与练习。不必先去背代码变量名。

02 从约束问题，一步一步得到拉格朗日形式

先看一个**教学用的标准问题**。θ 表示调度策略的参数：

$$\text{目标：让平均能耗} E(\theta) \text{ 尽量小；}$$
$$\text{要求：代价} C(\theta) \text{ 不超过预算} b.$$

把“不能超过”写成数学形式，再把右边移到左边：

$$C(\theta) \le b \quad \Longleftrightarrow \quad C(\theta) - b \le 0.$$

引入一个非负系数 $\lambda \ge 0$，构造拉格朗日函数：

$$\begin{aligned} \mathcal{L}(\theta, \lambda) &= E(\theta) + \lambda\big(C(\theta) - b\big) \\ &= E(\theta) + \lambda C(\theta) - \lambda b. \end{aligned}$$

为什么是加号？这里在最小化。当 $C > b$ 时，多出来的正项会让这个方案变差。若改成最大化收益，令 $R(\theta) = -E(\theta)$，就变成：

$$-\mathcal{L}(\theta, \lambda) = R(\theta) - \lambda C(\theta) + \lambda b.$$

固定 λ 时，λb 与动作、策略参数无关。因此，对策略来说，可以只比较 $R - \lambda C$。**但更新乘子时，预算 b 不能丢**。

策略与乘子，更新方向为什么相反？

对固定的乘子，策略尽量减小 $\mathcal{L}$；约束超标时，乘子往上调，给策略更大压力。标准对偶的定义是：

$$g(\lambda) = \inf_{\theta} \mathcal{L}(\theta, \lambda), \qquad \max_{\lambda \ge 0} g(\lambda).$$

不必急着求这个最优解。先注意：固定 θ，$\mathcal{L}$ 对 λ 的变化率就是 $C(\theta) - b$，所以常见的近似更新为：

$$\lambda_{k+1} = \max\{0, \lambda_k + \alpha(C_k - b)\}.$$

超标就增加，低于预算就减少。α 是每次调整的步幅。

> 这一页是标准理论的入门，不是说你的代码完整求解了这个对偶问题。你的实现使用不同层的代理代价、窗口统计、比例–积分和上限裁剪，应称为**拉格朗日启发的自适应惩罚**；不能由此宣称必然收敛或必然满足约束。理论出处见末页 [1]。

03 四个乘子，究竟在哪儿扣分？

上层：先算三个网络的值，再合成动作评分

对候选服务器 m，三个网络分别预测：能耗收益 $Q_E(m)$、Deadline 代价 $Q_D(m)$、可靠性代价 $Q_R(m)$。省略状态记号后：

$$S(m) = Q_E(m) - \mu_D^H \max\{0, Q_D(m)\} - \mu_R^H Q_R(m),$$
$$m^* = \arg\max_m S(m).$$

arg max 就是“选评分最高的服务器”。这里 **Deadline 预测做非负截断，可靠性预测没有同样截断**，按当前实现保留这一差别。

教学例子：可靠性预测暂设为 0，只看 Deadline 系数。

	A 服务器	B 服务器
Q_E	1.00	0.80
Q_D	0.20	0.02

当 $\mu_D^H = 0.5$：$S(A) = 1 - 0.5 \times 0.2 = 0.90$，$S(B) = 0.8 - 0.5 \times 0.02 = 0.79$，选 A。
当 $\mu_D^H = 3$：$S(A) = 1 - 3 \times 0.2 = 0.40$，$S(B) = 0.8 - 3 \times 0.02 = 0.74$，改选 B。

网络预测即使暂时不变，改变乘子也能改变选择。这里的预测值是训练尺度上的数，**不是直接拿 J 去减可靠性概率**。

下层：在 SAC 用于学习的奖励里扣分

先把省电收益和局部时间代价处理成可比较的尺度：

$$B_t = Z_E(\Delta E_i) - Z_h(h_t).$$

ΔE_i 是相对满频节省的能量；h_t 是降频消耗的局部时间裕量；Z 表示代码中的标准化、尺度下限和裁剪。随后：

$$r_t^L = B_t - \mu_D^L c_{D,t}^L - \mu_R^L c_{R,t}^L.$$

若 $B_t = 0.6, c_D^L = 0.05, c_R^L = 0.01, \mu_D^L = 1, \mu_R^L = 2$，则 $r_t^L = 0.6 - 1 \times 0.05 - 2 \times 0.01 = 0.53$。

不是直接“乘子变大就把频率改成某个值”。乘子改变训练奖励，奖励影响 Critic 和 Actor 的学习，随后策略的调频行为才可能改变。神经网络学习并不保证每一次动作都按同一方向变化。

04 上层 Deadline：先算清楚“整批失败”

你的系统有两个不同的 90%，需要分两层读：

1. $\Gamma = 0.90$：**一批内部**至少 90% 的工作流按时完成。
2. $\eta = 0.90$：希望**这样的批次**达标的概率至少 90%。

对第 e 批，先统计按时完成的工作流：

$$\mathrm{SR}_e = \frac{\text{该批按时完成数}}{\text{该批工作流总数}}.$$

再把它变成一个“这批是否失败”的 0/1 标记：

$$z_e = \begin{cases} 1, & \mathrm{SR}_e < 0.90, \\ 0, & \mathrm{SR}_e \geq 0.90. \end{cases}$$

例如训练一批 20 个工作流，18 个按时完成：$18/20 = 0.90$，$z_e = 0$；17 个按时完成：$17/20 = 0.85$，$z_e = 1$。测试一批 50 个时，45 个按时完成也算这批达标。

设窗口里已有 n 批，最多保留最近 60 批：

$$J_H = \frac{z_{e-n+1} + z_{e-n+2} + \cdots + z_e}{n},$$
$$b_H = 1 - \eta = 1 - 0.90 = 0.10,$$
$$\text{当前误差}\varepsilon_H = J_H - b_H = J_H - 0.10.$$

这里 J_H 是失败批次比例，不是平均工作流迟到比例。

最近 60 批中 12 批失败：$J_H = 12/60 = 0.20$。

$$\varepsilon_H = 0.20 - 0.10 = 0.10 > 0.$$

需要提高 Deadline 惩罚。若只有 3 批失败：$J_H = 3/60 = 0.05$，误差 $= -0.05$，当前压力可以减小。

上层前 50 个训练 episode 不更新 Deadline 乘子；之后也要有足够观测，当前实现至少需要 10 批。**60 是窗口容量，不是必须等满 60 批才能开始**。上述比例只是有限样本统计，不是总体成功率的保证。

05 比例–积分：把 PI 完全写开

PI 是**比例–积分**（Proportional–Integral）：

- 比例项 P：只看**现在**超标多少，立刻响应。
- 积分项 I：把**过去每次**的超标量逐步累积，保留记忆。

下面每个 k 都表示一次**允许执行的更新**。设当前统计量为 J_k、预算为 b、积分步幅为 τ、比例系数为 K_P、乘子上限为 M。你的代码依次做：

$$
\begin{aligned}
\text{第 1 步：误差} &\quad \varepsilon_k = J_k - b, \\
\text{第 2 步：积分候选} &\quad \widetilde{U}_{k+1} = U_k + \tau(J_k - b), \\
\text{第 3 步：积分裁剪} &\quad U_{k+1} = \min\{M, \max\{0, \widetilde{U}_{k+1}\}\}, \\
\text{第 4 步：比例项} &\quad P_k = K_P(J_k - b), \\
\text{第 5 步：输出候选} &\quad \widetilde{\mu}_{k+1} = U_{k+1} + K_P(J_k - b), \\
\text{第 6 步：乘子裁剪} &\quad \mu_{k+1} = \min\{M, \max\{0, \widetilde{\mu}_{k+1}\}\}.
\end{aligned}
$$

先把数限制到不小于 0，再限制到不大于 M。例如 $\min\{6, \max\{0, -0.3\}\} = 0$；输入 7 则输出 6。**比例项本身可以为负；不是每一步都截成正数**。

为了看见“积分记忆”，**暂时假设两处裁剪都没有触发**，且 $U_0 = 1$：

$$
\begin{aligned}
U_1 &= 1 + \tau\varepsilon_0, \\
U_2 &= 1 + \tau\varepsilon_0 + \tau\varepsilon_1, \\
U_3 &= 1 + \tau\varepsilon_0 + \tau\varepsilon_1 + \tau\varepsilon_2, \\
\mu_{k+1} &= 1 + \tau(\varepsilon_0 + \varepsilon_1 + \cdots + \varepsilon_k) + K_P\varepsilon_k.
\end{aligned}
$$

> 上面的累加展开**仅在没有触发裁剪时成立**；实际计算始终按前面的六步。积分状态 U 存过去，比例项只取当前误差。**下次累加用旧 U，不是旧乘子 μ**。PI 没有微分项，不应叫完整 PID。相关控制视角见末页 [2]。

06 上层 Deadline：把你的参数全部代进去

上层参数是：预算 0.10、积分步幅 0.10、比例系数 3、上限 6；初始 $U_H = \mu_D^H = 1$。每次有效更新完整写成：

$$U_{H,k+1} = \min\{6, \max\{0, U_{H,k} + 0.10(J_{H,k} - 0.10)\}\},$$
$$\mu_{D,k+1}^H = \min\{6, \max\{0, U_{H,k+1} + 3(J_{H,k} - 0.10)\}\}.$$

第一次：设已经过预热，窗口 60 批中 12 批失败，更新前积分状态为 1：

$$\begin{aligned} J_{H,0} &= 12/60 = 0.20, \\ \varepsilon_{H,0} &= 0.20 - 0.10 = 0.10, \\ U_{H,1} &= \min\{6, \max\{0, 1 + 0.10 \times 0.10\}\} = 1.01, \\ P_0 &= 3 \times 0.10 = 0.30, \\ \mu_{D,1}^H &= \min\{6, \max\{0, 1.01 + 0.30\}\} = 1.31. \end{aligned}$$

第二次：下一次窗口中 11 批失败，例如最旧的失败批次移出，新来的批次达标。**接着用刚才的** $U_{H,1} = 1.01$**：**

$$\begin{aligned} J_{H,1} &= 11/60 \approx 0.183333, \\ \varepsilon_{H,1} &= 11/60 - 0.10 \approx 0.083333, \\ U_{H,2} &= 1.01 + 0.10 \times 0.083333 \approx 1.018333, \\ P_1 &= 3 \times 0.083333 = 0.25, \\ \mu_{D,2}^H &= 1.018333 + 0.25 = 1.268333. \end{aligned}$$

两次都超标，**积分状态确实增加了**；但当前误差变小，比例项减少，所以实际乘子可以从 1.31 降至 1.268333。**“只要超标，乘子就一定比上次大”对 PI 并不成立。**

常见错误是第二次算成 $1.31 + 0.10 \times 0.083333 + 0.25$。这把旧比例项也当成积分状态，重复累加了。正确起点是 1.01。

07 下层 Deadline：先算降频究竟多造成多少迟到

下层不是简单地把“自己迟到”记为 1，而是看当前固定调度中，**降频额外增加的归一化迟到**。本页不要求你先学完整的责任分配算法。

对工作流 v，设相对 Deadline 为 D_v，实际响应时间为 M_v。先定义归一化迟到量：

$$\ell_v(M_v) = \max\{0, M_v/D_v - 1\}.$$

固定任务放置、服务器执行顺序和原始分配时刻，把整批任务都改成满频，重算响应时间 M_v^{all}。下层观测代价是：

$$d_v^L = \min\{1, \max\{0, \ell_v(M_v) - \ell_v(M_v^{\mathrm{all}})\}\}.$$

Deadline 为 100 秒，实际 130 秒，固定调度改满频后 110 秒：

$$\begin{aligned} \ell_v(M_v) &= \max\{0, 130/100 - 1\} = 0.30, \\ \ell_v(M_v^{\mathrm{all}}) &= \max\{0, 110/100 - 1\} = 0.10, \\ d_v^L &= \min\{1, \max\{0, 0.30 - 0.10\}\} = 0.20. \end{aligned}$$

满频也会迟到的那部分，没有在这里全部算到调频头上。

一批有 N_e 个工作流，先求这批的平均影响：

$$a_e = \frac{d_1^L + d_2^L + \cdots + d_{N_e}^L}{N_e}.$$

最近最多 40 个 a_e 按从大到小排列，取最大的 25% 再平均。若已有 n 批，取 $q = \max\{1, \lceil 0.25n \rceil\}$ 个；$\lceil\ \rceil$ 表示向上取整：

$$J_L = \frac{a_{(1)} + \cdots + a_{(q)}}{q}, \qquad a_{(1)} \geq \cdots \geq a_{(n)}.$$

满 40 批时就是最坏 10 批的均值。教学用 4 批 $0.01, 0.03, 0.06, 0.12$，最坏 25% 只取 0.12，所以 $J_L = 0.12$，不是四批平均值 0.055。

实际实现是有限窗口的“最坏 25% 均值”估计。它是尾部风险代理，不等同于总体分布上的精确 CVaR，也不是 ESP。

08 下层 Deadline：把 PI 及归零的情况写开

下层参数：预算 0.08、积分步幅 0.20、比例系数 25、上限 8。初始 $U_L = \mu_D^L = 1$。每次有效更新：

$$U_{L,k+1} = \min\{8, \max\{0, U_{L,k} + 0.20(J_{L,k} - 0.08)\}\},$$
$$\mu_{D,k+1}^L = \min\{8, \max\{0, U_{L,k+1} + 25(J_{L,k} - 0.08)\}\}.$$

例 A：已经过预热，$U_{L,0} = 1, J_{L,0} = 0.12$。

$$\begin{aligned}
\varepsilon_{L,0} &= 0.12 - 0.08 = 0.04,\\
U_{L,1} &= 1 + 0.20 \times 0.04 = 1.008,\\
P_0 &= 25 \times 0.04 = 1,\\
\mu_{D,1}^L &= \min\{8, \max\{0, 1.008 + 1\}\} = 2.008.
\end{aligned}$$

例 B：独立的另一种情形，更新前也设 $U_L = 1$，但 $J_L = 0$：

$$\begin{aligned}
\varepsilon_L &= 0 - 0.08 = -0.08,\\
U_L^{\text{new}} &= 1 + 0.20 \times (-0.08) = 0.984,\\
P &= 25 \times (-0.08) = -2,\\
\widetilde{\mu}_D^L &= 0.984 - 2 = -1.016,\\
\mu_D^{L,\text{new}} &= \min\{8, \max\{0, -1.016\}\} = 0.
\end{aligned}$$

预算 0.08 **不表示允许 8% 的工作流迟到**，也不表示 Deadline 缩短 8%。它限制的是上一页定义的尾部统计量。下层系数变成 0 可能是这个更新规则的直接结果，不一定是程序故障。

当 $\mu_D^L = 0$，下层奖励中的这一项 $\mu_D^L c_D^L$ 为 0，但局部时间项 $Z_h(h_t)$、上层 Deadline 代价、可靠性项并不因此全部消失。也不能仅凭归零就断言 Deadline 模块无用或有用。

下层前 20 个训练 episode 不更新 Deadline 乘子；之后至少要有 4 个有效批次统计。40 仍只是窗口容量。重新加载模型时控制器可能恢复已保存状态，不能总把实际 U 当成 1；这里的 1 只是从头训练的初值。

09 上层可靠性：非负缺口，越界才增加

用 R_w 表示工作流可靠性要求，用 $\mathcal{R}_w$ 表示实际模型可靠性。当前要求通常为 $R_w = 0.90$。上层看的不是一个简单的 0/1，而是**对数可靠性缺口**：

$$d_w^R = \max\{0, \ln R_w - \ln \mathcal{R}_w\}.$$

ln 是自然对数，可以用计算器的 ln 键。由于 ln 单调递增，实际值低于要求时，$\ln R_w - \ln \mathcal{R}_w$ 为正；满足要求时截为 0。

为什么可靠性会出现对数？若独立任务的可靠性相乘得到工作流可靠性：

$$\mathcal{R}_w = r_1 r_2 \cdots r_n, \qquad \ln \mathcal{R}_w = \ln r_1 + \ln r_2 + \cdots + \ln r_n.$$

乘积转成求和，便于累计和计算；这不代表独立性假设已经被现实数据验证。

上层可靠性没有这里介绍的额外比例项。按每个完成工作流的缺口更新一次，步幅 0.001、上限 100：

$$\mu_R^{H,\text{new}} = \min\{100, \max\{0, \mu_R^{H,\text{old}} + 0.001 \max\{0, \ln R_w - \ln \mathcal{R}_w\}\}\}.$$

训练收集完成后可以批量遍历这些工作流执行更新；**不是每个任务都更新一次**。

若旧系数为 1、要求 0.90、实际 0.85：

$$d_w^R = \ln(0.90/0.85) \approx 0.0571584,$$
$$\mu_R^{H,\text{new}} = 1 + 0.001 \times 0.0571584 \approx 1.00005716.$$

若下一个工作流实际可靠性为 0.95：缺口 $\max\{0, \ln(0.90/0.95)\} = 0$，系数**保持不变**，不是往下减。

结论：在这条规则下，上层可靠性乘子只增不减，达到上限后不能继续增大。“一切都达标就四个乘子一起下降”是不对的。

10 下层可靠性：看最近工作流的 0/1 失败标记

下层可靠性控制器改看每个已完成工作流是否达到要求：

$$v_w = \begin{cases} 1, & \mathcal{R}_w < R_w, \\ 0, & \mathcal{R}_w \geq R_w. \end{cases}$$

对最近最多 100 个工作流求均值。窗口中已有 n_R 个时：

$$J_R = \frac{v_1 + v_2 + \cdots + v_{n_R}}{n_R}.$$

预算 $b_R = 0.005$、步幅 0.5、上限 50；这条也没有额外 P 项：

$$\mu_R^{L,\text{new}} = \min\{50, \max\{0, \mu_R^{L,\text{old}} + 0.5(J_R - 0.005)\}\}.$$

满 100 个工作流的例子：

$$\begin{aligned} &1 \text{ 个失败：} J_R = 1/100 = 0.01, \\ &\qquad \mu_R^{L,\text{new}} = 1 + 0.5(0.01 - 0.005) = 1.0025; \\ &\text{全都达标：} J_R = 0/100 = 0, \\ &\qquad \mu_R^{L,\text{new}} = 1 + 0.5(0 - 0.005) = 0.9975. \end{aligned}$$

上面是两种独立情形，旧系数都取 1。$0.005 = 0.5\%$，不是 5%；窗口满 100 时，一个失败就是 1%，已经高于 0.5% 预算。**训练允许的小预算不等于测试表中把 Rel.=100% 改成 99.5%。**

务必分开：调乘子的统计量，与奖励里的代价

- 此处 J_R 用最终 0/1 标记，告诉控制器最近是否经常违反要求。
- SAC 奖励里的 $c_{R,t}^L$ 仍是降频额外引起的投影可靠性缺口，不是把每个任务都简单罚 1。

若投影可靠性在频率 f 下为 $\widehat{\mathcal{R}}_{w,t}(f)$，定义

$$\begin{aligned} u_t(f) &= \frac{\max\{0, \ln R_w - \ln \widehat{\mathcal{R}}_{w,t}(f)\}}{-\ln R_w}, \\ c_{R,t}^L &= \max\{0, u_t(f_i) - u_t(F_{m_i})\}. \end{aligned}$$

先算当前频率的缺口，再减去同一任务满频时的缺口；负差值截成 0。这就是为什么“奖励代价”和“控制器观测”不能混写成一个量。

11 把整套训练流程串起来，再自己算一遍

1. 用当前 μ_D^H, μ_R^H 给候选服务器打分，选择任务放置。
2. SAC 给出频率，环境实际执行，记录能耗、时刻和可靠性信息。
3. 一批结束，形成上层整体代价和下层额外代价；完成对应的反馈分配。
4. 上层可靠性按工作流缺口更新；神经网络用各自数据和损失学习。下层学习奖励使用当前乘子。
5. 轨迹结束时，满足门槛后更新上下层 Deadline 控制器及下层可靠性控制器；新的乘子供后续训练使用。实现中的各更新不是同时求出一个解析最优解。
6. 测试时冻结网络权重和这些乘子，不利用测试结果继续调它们。

自测题：先挡住答案，用计算器算

1. 上层 Deadline：已经允许更新，$U_H = 0.5, J_H = 0.15$。新积分量、比例项、乘子各是多少?
2. 下层 Deadline：已经允许更新，$U_L = 0.8, J_L = 0.04$。为什么乘子可能变成 0?
3. 上层可靠性：$\mu_R^H = 2$、实际可靠性等于要求 0.90。更新后是多少?
4. 下层可靠性：$\mu_R^L = 1$，最近 100 个工作流有 2 个失败。更新后是多少?

逐步答案

1. 误差 $0.15-0.10 = 0.05$；积分 $0.5+0.10\times0.05 = 0.505$；比例项 $3\times0.05 = 0.15$；输出 $0.505+0.15 = \boxed{0.655}$。

2. 误差 $0.04 - 0.08 = -0.04$；积分 $0.8 + 0.20 \times (-0.04) = 0.792$；比例项 $25 \times (-0.04) = -1$；候选值 $0.792 - 1 = -0.208$；裁剪后 $\boxed{0}$。

3. 缺口 $\max\{0, \ln 0.90 - \ln 0.90\} = 0$；输出 $2 + 0.001 \times 0 = \boxed{2}$。

4. 失败比例 $2/100 = 0.02$；误差 $0.02 - 0.005 = 0.015$；输出 $1 + 0.5 \times 0.015 = \boxed{1.0075}$。

能独立做对这四道题，就已经理解了四个乘子各自看什么、怎么调、能不能下降。之后再看源码，不必从神经网络内部开始找。

12 最后一页：参数速查、代码位置和结论边界

	上层 Deadline	下层 Deadline	上层可靠性	下层可靠性
观测	失败批次比例	额外迟到尾均值	对数缺口	失败工作流比例
预算	0.10	0.08	0	0.005
积分/调整步幅	0.10	0.20	0.001	0.5
额外比例系数	3	25	无	无
乘子上限	6	8	100	50
初始乘子	1	1	1	1
统计窗口	最近 60 批	最近 40 批最坏 25%	逐工作流	最近 100 工作流
预热	前 50 轮	前 20 轮	无此门槛	无额外预热

Deadline 的两个积分状态也从 1 开始。可靠性下层窗口非空即可统计，不必等满 100 个；以上均针对当前论文完整模型，不混用 Raw 或预算 0.01 的诊断版本。

代码导航：看函数，不必通读整仓库

- 上层 Deadline：`src/D3QN/deadline_lagrangian.py`。
  函数：`DeadlineLagrangianController.update`。
- 下层 Deadline：`src/sac/deadline_lagrangian.py`。
  函数：`LowDeadlineLagrangianController.update`。
- 上层可靠性：`src/D3QN/reliability_aware_high.py`。
  函数：`ReliabilityLagrangian.update`。
- 下层可靠性：`src/sac/reliability_lagrangian.py`。
  函数：`LowReliabilityLagrangianController.update`。
- 当前耦合训练入口：
  `tools/train_coupled_hierarchical_d3qn_sac.py`。
  函数 `install_low_dual_attribution` 将下层可靠性控制器的输入替换为最终工作流 0/1 标记；不要只读底层收集器就误认输入。
- 调用时机与下层 Deadline 统计构造：
  `tools/train_hierarchical_d3qn.py`。
  看轨迹末更新及函数 `_workflow_counterfactual_low_deadline_credit`。

三句最重要的话：乘子不是硬约束开关；不同预算对应不同统计量；有限样本达标不等于总体可行性证明。你的论文当前用“拉格朗日启发”是对实现边界的准确说明。

可继续阅读的原始资料

[1] Boyd、Vandenberghe，*Convex Optimization*，第 5 章 Duality。标准拉格朗日、对偶函数与非负乘子的定义：https://see.stanford.edu/materials/lsocoee364a/05Duality.pdf

[2] Stooke、Achiam、Abbeel，*Responsive Safety in Reinforcement Learning by PID Lagrangian Methods*，ICML 2020。用于理解乘子更新的控制视角，不意味着你的 PI 实现完整复现了其 PID 算法：https://proceedings.mlr.press/v119/stooke20a.html

本项目依据：当前 `paper/manuscript/main2.tex` 的约束反馈与乘子更新部分，以及上述控制器和 `configs/v3_sm_74_legacy_train.json`。讲义中的小数演算仅为教学，未改变模型、训练或实验数据。